

Sistema de Arquitetura Moderna para Simulações LVC

Rafael Rodrigues Pereira ¹

Fabio Kawaoka Takase ¹

¹Faculdade de Computação e Informática (FCI)
Universidade Presbiteriana Mackenzie São Paulo, SP – Brasil

[<rafaelrodrigues.pereira@mackenzista.com.br>](mailto:rafaelrodrigues.pereira@mackenzista.com.br)
[<fabio.takase@mackenzie.br>](mailto:fabio.takase@mackenzie.br)

2026

Resumo

*A complexidade inerente à gestão urbana no contexto das Smart Cities demanda soluções avançadas para o planejamento e suporte à decisão, visando elevar a consciência situacional e a resiliência dos ecossistemas urbanos. Este trabalho propõe e valida uma arquitetura de software orientada a eventos para uma plataforma que integra os domínios Live, Virtual e Constructive (LVC) sobre o Campus Living Lab da Universidade Presbiteriana Mackenzie (UPM). Fundamentada nos padrões Event Sourcing (ES) e Command Query Responsibility Segregation (CQRS), a arquitetura é construída exclusivamente sobre tecnologias Open Source, superando as barreiras de licenciamento e adoção dos padrões tradicionais HLA e DIS. O contrato canônico de interoperabilidade é definido via Protocol Buffers e gRPC, com geração automatizada de stubs em Java, Python e Rust pelo módulo **proto-shared**. A plataforma foi materializada em seis módulos independentes e exercitada por meio de adaptadores Live para AIS, SPTrans (OlhoVivo) e CGESP, um componente Virtual com interface Vaadin sobre o campus Higienópolis, e um componente Constructive baseado em AirSim com planejamento de trajetória apoiado por NVIDIA NIM e Ollama Gemma. Quatro cenários LVC integrados — cadastro de áreas e POIs, check-in/check-out de entidades, disparo de missão de drone por evento Live e auditoria de alertas — foram demonstrados sobre dois dashboards Grafana de observabilidade. Os resultados validam a viabilidade técnica da arquitetura em quatro dimensões: rastreabilidade, reprodutibilidade, interoperabilidade multi-linguagem e integração com fontes reais. Como limitações, a validação estatística comparativa entre dados Live e modelos Virtual/Constructive e a avaliação de escalabilidade horizontal sob carga massiva permanecem como trabalho futuro.*

Palavras-chave: Gêmeos Digitais; LVC; Arquitetura Orientada a Eventos; Event Sourcing; Open Source.

Abstract

*The inherent complexity of urban management in the context of Smart Cities demands advanced solutions for planning and decision support, aiming to increase situational awareness and resilience of urban ecosystems. This work proposes and validates an event-driven software architecture for a platform that integrates the Live, Virtual, and Constructive (LVC) domains over the UPM Campus Living Lab. Grounded on the Event Sourcing (ES) and Command Query Responsibility Segregation (CQRS) patterns, the architecture is built exclusively on Open Source technologies, overcoming the licensing and adoption barriers of traditional HLA and DIS standards. The canonical interoperability contract is defined via Protocol Buffers and gRPC, with automated generation of Java, Python, and Rust stubs by the **proto-shared** module. The platform was materialized as six independent modules and exercised through Live adapters for AIS, SPTrans (OlhoVivo) and CGESP, a Virtual component with a Vaadin interface over the Higienópolis campus, and a Constructive component based on AirSim with trajectory planning supported by NVIDIA NIM and Ollama Gemma. Four integrated LVC scenarios — registration of areas and POIs, entity check-in/check-out, drone mission triggered by Live events, and alert auditing — were demonstrated over two Grafana observability dashboards. Results validate the architecture's technical feasibility along four dimensions: traceability, reproducibility, multi-language interoperability, and integration with real sources. As limitations, the statistical comparative validation between Live data and Virtual/Constructive models, and the evaluation of horizontal scalability under massive load, remain as future work.*

Keywords: Digital Twins; LVC; Event-Driven Architecture; Event Sourcing; Open Source.

1 Introdução

A complexidade de gerir uma cidade ou uma parte significativa dela é um desafio que demanda suporte para planejar, operar, monitorar e controlar, buscando maior consciência situacional, desempenho, resiliência e previsibilidade frente aos desafios que se apresentam.

Originalmente utilizados na indústria para o desenvolvimento de produtos complexos (GRIEVES, 2014) os gêmeos digitais, recentemente se tornaram uma alternativa atrativa (MAZZETTO, 2024) nesta busca por uma consciência situacional acionável. Como modelos *live-virtual* que mimetizam sistemas físicos, incluindo seu ambiente circundante e processos operacionais, os gêmeos digitais estabelecem um fluxo contínuo de dados entre o mundo físico e o mundo virtual, podendo expandir a compreensão e potencializar a gestão de ecossistemas complexos (LV; SHANG; GUIZANI, 2022).

O uso efetivo de gêmeos digitais em áreas urbanas enfrenta obstáculos como a coleta de dados, conectividade entre sistemas, manipulação de grande volume de informação e integração e interoperabilidade entre diferentes tecnologias e sistemas (MAZZETTO, 2024) e a visualização de cenários e dados aumentando a consciência situacional para planejamento e tomada de decisões.

A simulação construtiva também é utilizada para enriquecimento de cenários com eventos raros e/ou de alta periculosidade em campanhas de treinamento ou em testes de sistemas (NOSEWORTHY, 2008; THEUNISSEN; KOTEGAWA, 2017), podendo simular inclusive comportamentos de grupos de pessoas (LOPER; SITTERLE, 2023). Estes recursos também podem ser aplicados no contexto urbano em cenários de interesse da defesa civil e de segurança pública.

Este trabalho tem como objetivo atender uma demanda urgente de desenvolvimento tecnológico (DANI et al., 2023) ao propor uma arquitetura de sistemas para uma plataforma que suporte a integração e interoperabilidade entre sistemas de coleta de dados, armazenamento, modelos digitais, soluções de visualização e de simulação tendo como núcleo o campus de Higienópolis da UPM, suportando atividades realizadas no projeto de pesquisa de Gêmeos Digitais da FCI e o programa de desenvolvimento tecnológico e de parcerias da FCI denominado Campus Living Lab.

2 Referencial Teórico

A simulação, técnica de modelagem que busca representar e reproduzir o comportamento dinâmico de um sistema real ou conceitual ao longo do tempo (BANKS, 1998), é uma ferramenta essencial para análise, projeção e otimização de sistemas. Sua aplicação oferece um ambiente controlado para experimentação e tomada de decisão, sendo fundamental na resolução de problemas do mundo real. Com o avanço tecnológico, a simulação evoluiu de uma ferramenta preditiva para um ambiente de experimentação e treinamento mais abrangente, buscando maior realismo e capacidade de integração. Essa busca por interoperação de modelos é crucial devido aos requisitos de reuso, escalabilidade e, por fim, aos requisitos dos resultados esperados de treinamentos e capacitação (KIM et al., 2014).

Em contextos urbanos complexos, como o de *Smart Cities*, a simulação se materializa por meio do conceito de Gêmeo Digital (*Digital Twin*). O Gêmeo Digital atua como uma representação virtual fiel e dinâmica de um sistema físico (a cidade ou um subsistema urbano), integrado a dados reais. Este artefato virtual permite realizar previsões, análises e, crucialmente, simulações de cenários, suportando a tomada de decisões urbanas por meio da combinação entre o real e o virtual. Diante da crescente complexidade dos sistemas contemporâneos, que exigem a validação de modelos contra dados reais e a simulação de cenários perigosos ou impraticáveis no mundo físico, a simulação tradicional se tornou insuficiente. Essa necessidade impulsionou o desenvolvimento de abordagens mais sofisticadas, tal como o paradigma *Live, Virtual e Constructive* (LVC). O LVC é um objetivo maior no campo de Modelagem e Simulação (M&S) para criar uma arquitetura de simulação integrada e unificada (GAO et al., 2021). LVC representa a integração de três tipos de recursos de simulação:

- *Live*: Pessoas reais operando sistemas reais.
- *Virtual*: Pessoas reais operando sistemas simulados.
- *Constructive*: Pessoas simuladas operando sistemas simulados.

Esta integração visa criar um ambiente coeso e interativo para simulações, embora alcançar a interoperabilidade LVC continue sendo um desafio considerável para a comunidade de M&S (KIM et al., 2014).

O paradigma LVC visa maximizar o realismo e a utilidade da simulação, permitindo que esses três componentes coexistam de forma sobreposta e interajam em um único cenário simulado. Essa integração proporciona uma transição contínua entre as várias fases de teste (THEUNISSEN; KOTEGAWA, 2017). No contexto de *Smart Cities*, a fusão LVC é particularmente robusta, pois permite a validação de modelos *Virtual* contra a realidade *Live* e a exploração segura de cenários *Constructive* de alto risco (como desastres ou picos

de demanda). Dentre suas diversas aplicações, o LVC é empregado para a modelagem e simulação de cenários complexos de combate (KIM et al., 2014), teste, avaliação e treinamento de Ciberoperações (VARSHNEY; PICKETT; BAGRODIA, 2011) e garantia da segurança e proteção de veículos autônomos complexos por meio da avaliação de reações a diversas situações em mundos virtualmente construídos (POTTEIGER et al., 2023).

A interoperabilidade entre os componentes é crucial para o sucesso de uma simulação LVC, pois deve garantir o fluxo preciso de dados e a consistência dos dados de diferentes fontes LVC para o cliente (THEUNISSEN; KOTEGAWA, 2017).

Para sustentar essas simulações integradas, arquiteturas como a *High Level Architecture* (HLA) e o *Distributed Interactive Simulation* (DIS) são frequentemente utilizadas, representando marcos na história da arquitetura de simulação distribuída (GAO et al., 2021). A HLA provê a base estrutural para a interoperabilidade da simulação (DAHMANN, 1997), com o propósito de reuso e interoperação de simulações. Já o DIS é um protocolo de comunicação focado na troca de dados entre simuladores em tempo real (GAO et al., 2021). A modularidade é um fator-chave nesse ambiente, pois permite separar funcionalidades com diferentes ciclos de projeto e avaliação, facilitando a manutenção e a evolução do ambiente LVC (THEUNISSEN; KOTEGAWA, 2017).

Entretanto, HLA e DIS apresentam limitações significativas no contexto de pesquisa acadêmica e desenvolvimento aberto. A HLA exige o uso de uma *Run-Time Infrastructure* (RTI), frequentemente proprietária e de elevado custo de licenciamento, representando uma barreira de entrada considerável para laboratórios e universidades (GAO et al., 2021). Além disso, a curva de aprendizado acentuada, a dependência de ferramentas altamente especializadas e a baixa integração com o ecossistema moderno de software aberto limitam a adoção ágil desses padrões em projetos de pesquisa (DANI et al., 2023). Essa realidade motiva a busca por arquiteturas alternativas que ofereçam interoperabilidade comparável com menor custo, maior acessibilidade e plena compatibilidade com tecnologias *Open Source*.

Diante desse desafio, o foco se volta para as arquiteturas de software capazes de suportar a interoperabilidade de alto desempenho e a escalabilidade massiva. Tais ambientes exigem que a arquitetura seja reativa, no sentido de ser *message-driven*, elásticas, resiliente e responsiva, conforme as metas do *Reactive Manifesto* (DEBSKI et al., 2018).

Nesse contexto de sistemas distribuídos e reativos, padrões de arquitetura como o *Event Sourcing* (ES) e o *Command Query Responsibility Segregation* (CQRS) surgem como soluções robustas. O CQRS (LONG, 2017) propõe a segregação de responsabilidade, separando o processamento de comandos (solicitações de escrita que modificam o estado do sistema) do processamento de *queries* (solicitações de leitura que não modificam o estado) (LJUBISAVLJEVIĆ et al., 2024). Já o *Event Sourcing* é um padrão fundamental, frequentemente utilizado em conjunto com o CQRS (DEBSKI et al., 2018), no qual o estado da aplicação é armazenado como uma sequência de eventos imutáveis (OVEREEM; SPOOR; JANSEN, 2017). Juntos, esses padrões fornecem a base para a criação de sistemas altamente disponíveis e escaláveis.

Para viabilizar a comunicação eficiente e multi-linguagem entre os componentes heterogêneos da plataforma, adota-se o *Protocol Buffers* (ProtoBuf) como Interface Definition Language (IDL) e o gRPC como protocolo de comunicação remota com payload binário de alto desempenho (SILVA et al., 2026). O ProtoBuf, desenvolvido pela Google, é um mecanismo de serialização binária agnóstico de linguagem e plataforma: a partir de um arquivo de contrato `.proto`, gera automaticamente código de serialização e deserialização

em múltiplas linguagens, garantindo um contrato de dados unificado e fortemente tipado entre os serviços. O gRPC, construído sobre HTTP/2, oferece comunicação síncrona e assíncrona de baixa latência com suporte nativo a *streaming* bidirecional. A combinação ProtoBuf + gRPC estabelece o contrato canônico de interoperabilidade da plataforma LVC, permitindo que componentes implementados em linguagens distintas (Java, Python e Rust) se comuniquem de forma transparente, desacoplada e com *overhead* mínimo de rede.

Complementarmente, a arquitetura adota padrões reativos consagrados para aumentar a resiliência e a responsividade em ambientes distribuídos. O padrão *Hexagonal Architecture* (ports & adapters) permite a separação clara entre lógica de negócio e detalhes de infraestrutura, facilitando testes e evolução incremental. O *Circuit Breaker* (via Resilience4j) protege contra falhas em cascata ao detectar indisponibilidade de serviços externos e aplicar backoff automático. *Virtual Threads* (Java 21+) reduzem o overhead de context-switching, permitindo alta concorrência com baixa alocação de memória. Esses padrões, em conjunto com o *Reactive Manifesto*, garantem que a plataforma seja *message-driven*, resiliente e responsiva mesmo sob carga elevada.

3 Metodologia

3.1 Definição do Problema e Objetivos do Trabalho

O problema central desta pesquisa reside na aplicação do paradigma *Live, Virtual e Constructive* (LVC) em contextos de *Smart Cities*, onde a ausência de uma plataforma interoperável e escalável limita o suporte à tomada de decisão no contexto urbano. O objetivo principal deste trabalho será realizar a pesquisa aplicada que suporte um estudo de viabilidade técnica na definição e construção de uma arquitetura de *software* proposta. Para atingir este objetivo, o trabalho foi estruturado em 4 etapas: (i)revisão da Literatura, para a construção da fundamentação teórica; (ii)definição da Arquitetura da Solução, focando nos padrões de design e contrato de interoperabilidade; (iii)implementação, para construção do artefato (plataforma LVC) em escala de viabilidade, e; (iv)validação da Plataforma, avaliação da capacidade da solução implementada na garantia da rastreabilidade, reprodutibilidade e da facilidade de integração LVC.

3.2 Design e Decisões Arquiteturais

A arquitetura de *software* é o artefato central deste trabalho. A escolha do modelo *Event-Driven* não é acidental: sistemas LVC são, por natureza, reativos: eventos do mundo real (*Live*) disparam atualizações em modelos digitais (*Virtual*), que por sua vez podem acionar simulações (*Constructive*). Uma arquitetura orientada a eventos espelha essa dinâmica, ao contrário de arquiteturas síncronas request-response que introduziriam acoplamento temporal e pontos únicos de falha.

Adicionalmente, os padrões tradicionais de interoperabilidade para simulação distribuída (HLA e DIS), embora consolidados no domínio militar, impõem barreiras significativas à pesquisa acadêmica: dependência de RTI proprietárias, curvas de aprendizado acentuadas e baixa integração com o ecossistema moderno de software aberto (DANI et al., 2023). A arquitetura aqui proposta substitui esses padrões por um conjunto de tecnologias *Open Source* maduras e amplamente adotadas na indústria de sistemas distribuídos, mantendo as garantias de interoperabilidade com custo zero de licenciamento e plena compatibilidade com ferramentas modernas de desenvolvimento, containerização e observabilidade.

O escopo é delimitado ao Estudo de Viabilidade Técnica, utilizando sistemas típicos de simulação em escala de teste.

3.2.1 Especificação da Solução

Contrato de Comunicação Canônico: A interoperabilidade será garantida pela padronização da comunicação via Barramento de Eventos (*Event Hub*) e Protocol Buffers (IDL), que juntos estabelecem um contrato de dados unificado para todos os componentes LVC. O *Event Hub*, implementado em *Rust*, atua como *broker* central de alto desempenho implementando o padrão *Publish/Subscribe* com suporte a *streaming* gRPC bidirecional.

Baixa Latência e Comunicação Assíncrona: Todos os componentes (*Live*, *Virtual* e *Constructive*) se comunicam exclusivamente via gRPC com o *Event Hub*, garantindo eficiência, baixa latência e transparência de localização. O protocolo HTTP/2 permite multiplexação nativa, reduzindo *overhead* de rede, enquanto *streaming* gRPC bidirecional nativo permite consumo em tempo real de eventos sem necessidade de polling.

Geração de Contratos Multi-Linguagem (proto-shared): Um módulo dedicado automatiza a compilação do IDL (*Protocol Buffers*), gerando *stubs* de comunicação em múltiplas linguagens (Java, Python, Rust) a partir de um único arquivo *.proto*, garantindo a padronização e a consistência das interfaces de integração entre os componentes LVC.

Validação Contínua: Foram implementados mecanismos de Validação Contínua (comparação *Virtual* vs. *Live*) para garantir a fidelidade e o ajuste dos modelos de simulação em relação aos dados reais.

Estratégia Open Source: O projeto foi desenvolvido e publicado integralmente como *Open Source*, e a viabilidade técnica foi demonstrada pela utilização e integração de ferramentas e soluções abertas.

3.2.2 Princípios Arquiteturais

Cada padrão arquitetural adotado responde diretamente a um requisito do domínio LVC e a uma limitação concreta das abordagens tradicionais.

O padrão *Event Sourcing* (ES) representa o estado dos *Gêmeos Digitais* como uma sequência de *eventos imutáveis* (*Event Store*). Esta escolha atende diretamente aos requisitos de rastreabilidade (cada mudança de estado é registrada como um evento auditável) e de reprodutibilidade (qualquer cenário passado pode ser reconstruído via *replay* integral da sequência de eventos).

O padrão CQRS (*Command Query Responsibility Segregation*) segrega as responsabilidades de escrita (*Comandos*) e leitura (*Consultas/Projeções*). No contexto LVC, o *write path* lida com ingestão de alta frequência de eventos *Live* e *Constructive*, priorizando durabilidade e atomicidade (Transactional Outbox); o *read path* serve dashboards (Grafana) e consultas do *Virtual*, priorizando latência de consulta (projeções materializadas).

Os princípios reativos (*Reactive Manifesto*) foram implementados pela arquitetura ser *message-driven*, elástica e resiliente. Picos de eventos, como rajadas de múltiplos simuladores em cenários *Constructive*, são absorvidos via *back-pressure* no barramento e escalonamento elástico dos consumidores, evitando degradação em cascata.

A escalabilidade foi obtida utilizando *Virtual Threads* e *Scoped Values*. A *Virtual Threads* (JEP 444) é o mecanismo central de concorrência nos componentes JVM. Gerenciadas em *user-mode* pela JVM, permitem milhões de threads leves com custo mínimo de

memória, eliminando o gargalo de *thread-per-request* nas múltiplas conexões gRPC simultâneas dos adaptadores *Live*. Complementarmente, *Scoped Values* (JEP 506) propagam contexto (*trace IDs*, identificadores de simulação) entre *Virtual Threads* de forma segura.

A Arquitetura Hexagonal (*Ports & Adapters*) promove a separação entre lógica de domínio e infraestrutura isolando cada fonte de dados externa em um *adapter* que implementa uma *port* definida pelo domínio. Isso permite testar a lógica de negócio isoladamente e integrar novas fontes de dados sem impacto no núcleo da aplicação — propriedade essencial para um *Living Lab* onde fontes urbanas são continuamente adicionadas.

3.3 Estudo de Ferramentas e Fontes de Dados

A demonstração de viabilidade técnica exige evidência de que a arquitetura comporta fontes heterogêneas e simuladores distintos. Por esse motivo, o trabalho não foi orientado a um cenário único de aplicação, mas sim à integração de fontes de dados diversas (APIs públicas, *web scraping*, dados geoespaciais abertos e simuladores) capazes de exercitar o contrato de interoperabilidade e validar o comportamento *event-driven* da plataforma sob entradas reais.

Todos os dados utilizados neste estudo são coletados de fontes públicas e gratuitas, alinhando-se ao caráter estritamente *Open Source* da arquitetura. O acesso à API OlhoVivo (SPTrans) é realizado mediante cadastro aberto e gratuito; o acesso à API AIS via aisfriends.com também é livre; os dados do CGESP são obtidos de informação pública disponibilizada pela Prefeitura de São Paulo, com frequência de consulta não-abusiva; e os dados geoespaciais do OpenStreetMap são de uso livre.

Para suportar essa abordagem foram estudadas as seguintes ferramentas, cada uma justificada pelo papel que exerce no ciclo LVC:

- *OpenStreetMap* ([OpenStreetMap contributors, 2026](#)): base de dados geoespacial aberta, utilizada para delimitar áreas de patrulha e pontos de interesse no campus Higienópolis, fornecendo insumo ao componente *Virtual*.
- *AirSim* ([Microsoft Corporation, 2022](#)): simulador de voo de drones em ambiente urbano fotorrealista, adotado como base *Constructive* para exercitar missões simuladas no mundo virtual.
- *AIS* (*Automatic Identification System*) via aisfriends.com ([AIS Friends, 2025](#)): API pública de rastreamento de embarcações em tempo real, adotada como uma das fontes *Live* de mobilidade marítima.
- *OlhoVivo* (SPTrans) ([São Paulo Transporte S/A,](#)): API pública de posição de ônibus em São Paulo, adotada como uma das fontes *Live* de mobilidade urbana.
- *CGESP* ([Centro de Gerenciamento de Emergências \(CGE\),](#)): portal público da Prefeitura de São Paulo com informação de alagamentos, integrado por *web scraping* como fonte *Live* de eventos de defesa civil.
- *protobuf-maven-plugin* (ascpes) ([SCOPES, 2026](#)): plugin Maven de compilação de IDL *Protocol Buffers*, responsável pela geração automatizada dos *stubs* Java e Python no módulo *proto-shared*.

- *Deepwiki* (Cognition Labs, 2025): plataforma de documentação automatizada para repositórios *Open Source*, adotada para publicar a documentação da plataforma de forma aberta e navegável.

A escolha deliberada por múltiplas fontes e ferramentas, em vez de um cenário monolítico, é, em si, parte da validação: comprovar que o contrato canônico e o barramento *event-driven* comportam a integração *plug-and-play* de naturezas distintas de dado.

4 Resultados e Discussões

4.1 Visão Geral da Arquitetura

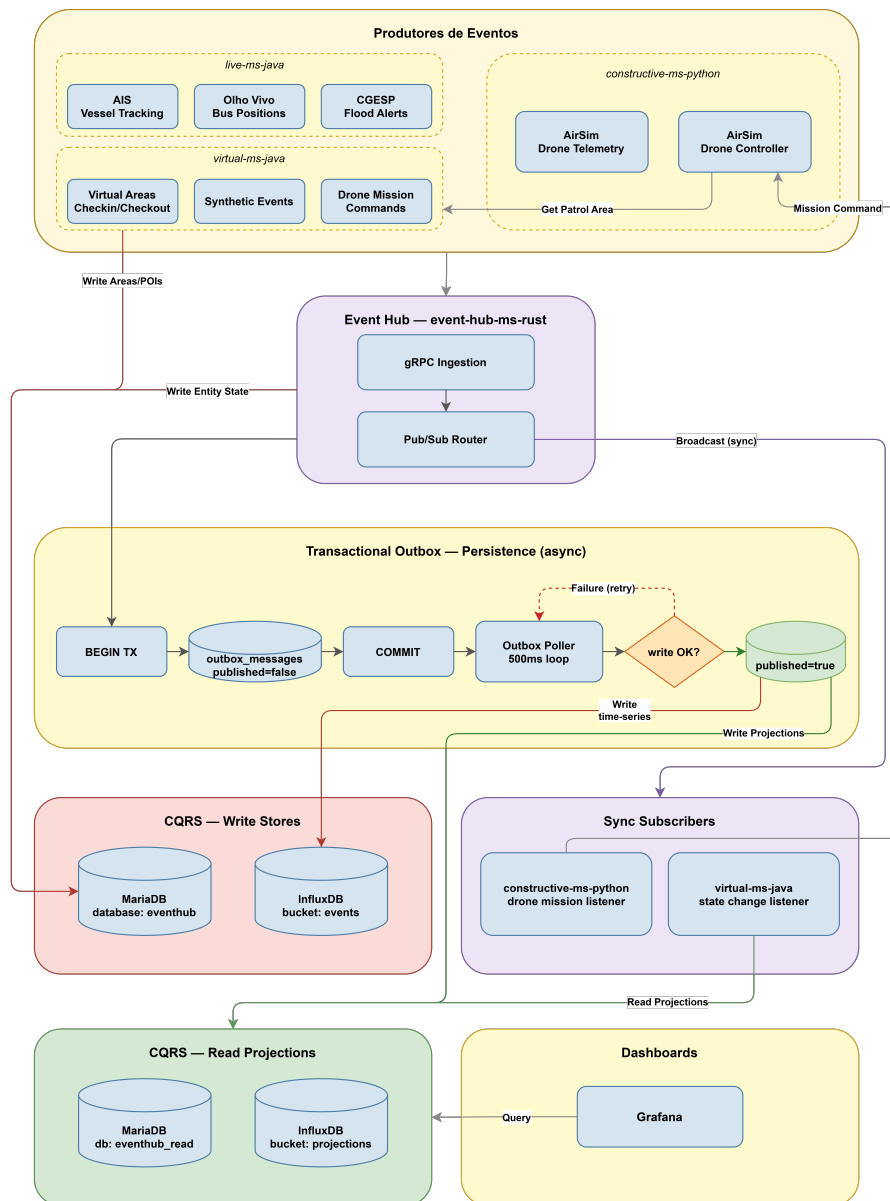


Figura 1 – Arquitetura *event-driven* da plataforma LVC.

A plataforma LVC cuja arquitetura é apresentada na Figura 1 é composta por seis módulos independentes, cada um implementado na linguagem e *stack* tecnológica mais adequada ao seu domínio, unificados pelo contrato canônico de comunicação via *Protocol Buffers* e gRPC. Esta abordagem poliglota é deliberada: em vez de forçar todos os componentes a uma única linguagem (como ocorre em implementações típicas de HLA, que exigem bindings específicos de C++ ou Java), cada módulo utiliza o ecossistema mais produtivo para sua função. O **proto-shared** atua como ponto único de verdade contratual, automatizando a geração de código cliente/servidor em Java (via Maven), Python (via ascpes **protobuf-maven-plugin**) e Rust (via imagem Docker dedicada com **protoc-gen-prost** e **protoc-gen-tonic**), eliminando a barreira de integração multi-linguagem. A Tabela 1 sumariza as tecnologias de cada componente.

Tabela 1 – Visão geral dos componentes da plataforma LVC e suas *stacks* tecnológicas.

Projeto	Linguagem	Framework	Tecnologias-Chave
constructive-ms-python	Python 3.11+	AirSim + FastAPI	NVIDIA NIM APIs, Ollama Gemma 4, gRPC, Protocol Buffers
event-hub-ms-rust	Rust	gRPC + HTTP/2	Pub/Sub bidirecional, MariaDB/InfluxDB, Docker, Outbox Pattern
infra-local-shared	Docker Compose	—	InfluxDB 2.7, MariaDB 11.4, HashiCorp Vault 1.17, Grafana
live-ms-java	Java 25	Spring Boot 4	Hexagonal Architecture, Resilience4j, Virtual Threads, Feign, Jsoup
proto-shared	—	Apache Maven + Docker	Protocol Buffers 4.34, gRPC 1.79, ascpes protobuf-maven-plugin (Java + Python), Docker codegen (Rust)
virtual-ms-java	Java 25	Spring Boot 4 + Vaadin 25.1	gRPC, MariaDB, InfluxDB, CQRS Write/Read

O fluxo de comunicação *event-driven* entre os componentes segue o paradigma CQRS com segregação explícita entre os caminhos de escrita (*Write*) e leitura (*Read*):

- **CQRS-Write:** Os componentes *Live* (**live-ms-java**) e *Constructive* (**constructive-ms-python**) publicam eventos no *Event Hub* (**event-hub-ms-rust**) via gRPC. O *Event Hub* persiste os eventos no InfluxDB (*time-series*) e na MariaDB (*outbox relacional*), garantindo atomicidade via *Transactional Outbox Pattern*. O componente *Virtual* (**virtual-ms-java**) escreve áreas de patrulha e pontos de interesse (POIs) diretamente na MariaDB (CQRS-Write).
- **CQRS-Read:** O *Virtual* consulta a base de projeções (**eventhub_read**) na MariaDB e *buckets* de projeção no InfluxDB para obter o estado materializado dos gêmeos digitais. O Grafana consome os dados do InfluxDB (*bucket projections*) e da MariaDB (**eventhub_read**) para gerar dashboards em tempo real.
- **Comunicação entre componentes:** O *Constructive* subscreve eventos de missão de drone no *Event Hub* e consulta o *Virtual* via gRPC para obter áreas de patrulha. O *Virtual*, por sua vez, subscreve eventos do *Event Hub* para detectar alterações de estado e publica novos eventos de volta ao barramento.

O código-fonte de todos os componentes está documentado no Deepwiki (<<https://deepwiki.com/tcc-modern-lvc-arch>>) e publicado integralmente como *Open Source* em um repositório no GitHub (<<https://github.com/tcc-modern-lvc-arch>>).

4.2 Implementação dos Componentes

A implementação segue uma abordagem modular e poliglota. Cada componente foi projetado para ser desenvolvido, testado e implantado independentemente — uma propriedade essencial para viabilizar a integração plug-and-play de novos Gêmeos Digitais e simuladores sem recompilar ou reconfigurar o barramento central.

4.2.1 Componente *Live* — live-ms-java

O componente *Live* é implementado como um microsserviço Spring Boot 4 (Java 25) utilizando *Hexagonal Architecture* (ports & adapters). Integra três fontes de dados geoespaciais em tempo real:

- AIS: rastreamento de embarcações via API aisfriends.com (AIS Friends, 2025), com integração via Feign Client + Resilience4j Circuit Breaker.
- OlhoVivo (SPTrans): posições de ônibus via API proprietária da SPTrans, com autenticação lazy e session management.
- CGESP: mapeamento de alagamentos (pontos de inundação) via web scraping (Jsoup) com parsing de dados HTML estruturados.

Cada adaptador implementa um padrão de *polling* agendado (@Scheduled) e publica eventos via gRPC para o *Event Hub*. Padrões reativos incluem: *Circuit Breaker* (Resilience4j) para falhas de API, *Virtual Threads* (Java 25) para alta concorrência com baixo overhead, e sealed classes + records (Java 25) para modelagem de domínio imutável e type-safe, e *Scoped Values* (JEP 506) para propagação de contexto entre *virtual threads*.

Todos os adaptadores implementados consomem exclusivamente fontes públicas. O acesso à API OlhoVivo (SPTrans) é realizado mediante cadastro aberto e gratuito, sendo os dados disponibilizados como informação pública sem possibilidade de comercialização. Os dados do CGESP são obtidos de fontes de informação públicas disponibilizadas pela Prefeitura de São Paulo, com frequência de consulta não-abusiva.

4.2.2 Componente *Constructive* — AirSim com IA

O componente *Constructive* é implementado em Python e integra o simulador AirSim para simulação de voo de drones em ambiente urbano. O planejamento de trajetória e a tomada de decisão do drone utilizam APIs gratuitas do NVIDIA NIM (NVIDIA Inference Microservices), que disponibiliza modelos de IA pré-treinados para visão computacional e controle de trajetória sem custo, evitando sacrificar a performance da máquina local com inferência pesada. Como alternativa local *offline*, o projeto também integra o modelo Ollama Gemma 4 (versão `gemma4:e2b`) para inferência de linguagem natural no planejamento de missões. O simulador publica eventos de posição, sensores e estado do drone via gRPC para o *Event Hub*.

4.2.3 Componente *Virtual* — Gêmeo Digital

O componente *Virtual* é implementado como um microserviço Spring Boot 4 (Java 25) com interface Vaadin 25.1, materializando o Gêmeo Digital do campus Higienópolis. O módulo gerencia áreas de patrulha (zonas georreferenciadas para o drone simulado), pontos de interesse (POIs) e o estado de entidades urbanas monitoradas pela plataforma. O CQRS se materializa em dois caminhos segregados: o *write path* persiste comandos e atualizações de estado na base *eventhub* (MariaDB), enquanto o *read path* consome projeções materializadas da base *eventhub_read* (MariaDB) e *buckets* de projeção do InfluxDB para alimentar a interface Vaadin e os dashboards do Grafana. O *Virtual* também subscreve eventos do *Event Hub* para detectar alterações de estado nos componentes *Live* e *Constructive*, publicando novos eventos de volta ao barramento conforme o estado do Gêmeo Digital é atualizado.

4.2.4 Barramento de Eventos e Persistência

- *Event Hub* (Rust + gRPC): barramento central implementando Pub/Sub bidirecional com suporte a *streaming* gRPC e Protocol Buffers como contrato de dados. Implementa o padrão *Transactional Outbox* para garantir entrega confiável de eventos: cada mensagem recebida é persistida atômica na MariaDB (tabela *outbox_messages*) e publicada de forma assíncrona para os *subscribers*, com retry automático em caso de falha de rede.
- Containerização: Docker Compose com rede *shared-infra-network* unificando *Event Hub*, *live-ms-java*, *virtual-ms-java* e simulador AirSim, garantindo portabilidade e reprodutibilidade.
- Persistência de Eventos (CQRS-Write): InfluxDB como banco de dados de séries temporais para armazenamento de eventos LVC com retenção configurável por bucket (*events* para escrita, *projections* para leitura). MariaDB com duas bases segregadas: *eventhub* (CQRS-Write) para registro de entidades, estado e *outbox*, e *eventhub_read* (CQRS-Read) para projeções materializadas consumidas pelo Grafana.
- Vault: HashiCorp Vault para gerenciamento centralizado de secrets (tokens de API, credenciais) com padrão *.env* para configuração local.

4.3 Fonte de Dados Geoespaciais

O ambiente urbano simulado pelo componente *Constructive* e modelado pelo Gêmeo Digital *Virtual* utiliza dados geoespaciais do OpenStreetMap. A Figura 2 apresenta o polígono correspondente à área do campus Higienópolis da UPM, utilizado como zona de patrulha para o drone simulado e como região de interesse para o *virtual-ms-java*.

4.4 Cenários de Interação LVC

A plataforma integrada foi exercitada através de cenários que conectam os três domínios LVC em um fluxo coeso, ilustrando como dados reais (*Live*), modelos digitais (*Virtual*) e simulações (*Constructive*) interoperam sobre o barramento *event-driven*.

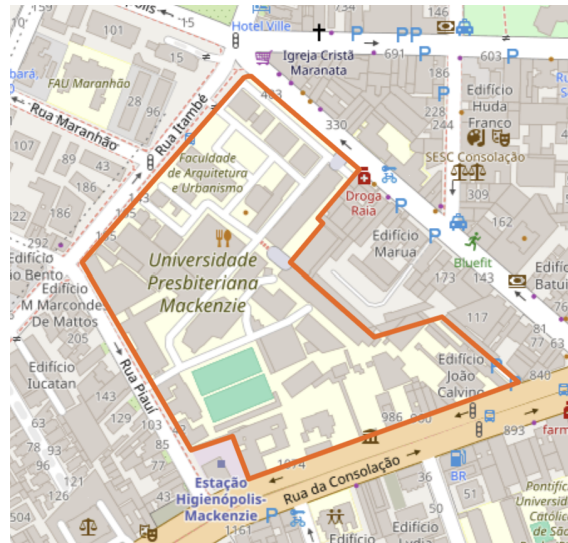


Figura 2 – Polígono do campus Higienópolis extraído do OpenStreetMap.

4.4.1 Cadastro de Áreas e Pontos de Interesse (*Virtual*)

O ponto de partida do ciclo é o cadastro, via SPA Vaadin do `virtual-ms-java`, de áreas georreferenciadas (zonas de patrulha) e pontos de interesse (POIs) sobre o campus Higienópolis. Esses artefatos passam a compor o estado do Gêmeo Digital: áreas atuam como regiões de monitoramento para entidades *Live*, e POIs como destinos candidatos a missões *Constructive*. A Figura 3 apresenta a tela de cadastro de áreas.

Areas
Points of Interest
Simulation
Monitoring

Virtual Areas

New Area

Name	Type	Monitors	Action	Status	Actions
Mackenzie Campus	POLYGON	DRONE	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Rua da Consolação	CORRIDOR	BUS	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Consolação Neighborhood	POLYGON	BUS	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete
Caio Prado C/B Stop Area	CIRCLE	BUS	MISSION_TRIGGER	Active	Edit Deactivate Delete
Santos Port	CIRCLE	VESSEL	CHECKIN_CHECKOUT	Active	Edit Deactivate Delete

Figura 3 – CRUD de áreas de patrulha na SPA do `virtual-ms-java`.

4.4.2 Ingestão *Live* e Eventos de *Check-in/Check-out*

Os adaptadores *Live* (AIS, OlhoVivo e CGESP) publicam continuamente no *Event Hub* eventos de posição de entidades reais. O *Virtual* subscreve esse fluxo e, a cada atualização, avalia se a entidade entrou ou saiu de uma área cadastrada, emitindo eventos de *check-in* e *check-out*. Esse casamento entidade-área é o mecanismo pelo qual fontes *Live* se tornam observáveis no Gêmeo Digital, sem acoplamento direto entre adaptador e modelo.

4.4.3 Missão do Drone (*Constructive*) Disparada por Evento *Live*

Quando uma entidade *Live* atinge um POI (por exemplo, um ônibus registrado no ponto *Caio Prado C/B* pela API OlhoVivo), o *Virtual* publica um evento de *entidade-em-POI*. O *constructive-ms-python* subscreve esse evento, consulta o *Virtual* via gRPC para obter as coordenadas da missão e dispara o voo simulado no AirSim, com planejamento de trajetória apoiado pelo NVIDIA NIM (ou alternativa local *offline* com Ollama Gemma). Ao alcançar o ponto, o drone captura fotos e publica os artefatos de volta no barramento, onde são auditados. O mesmo mecanismo pode ser acionado por eventos sintéticos gerados pela SPA (Figura 4), viabilizando reprodução controlada do cenário.

The screenshot shows the 'Virtual Areas' interface with a sidebar on the left containing 'Areas', 'Points of Interest', 'Simulation' (selected), and 'Monitoring'. The main panel is titled 'Virtual Areas' and contains two sections: 'Simulation' and 'Simulate Virtual Flood Event'. The 'Simulation' section has a 'Ping Virtual Move Event' form with fields for 'Entity ID' (e.g. vehicle-001), 'Entity Type' (BUS), 'Latitude', 'Longitude', 'Altitude m (optional)', and 'LVC Origin' (VIRTUAL). A 'Send Move Event' button is below. The 'Simulate Virtual Flood Event' section has fields for 'Area', 'Severity' (moderado), 'Water Level (cm)' (50), and 'LVC Origin' (VIRTUAL). A 'Simulate Flood' button is at the bottom.

Figura 4 – Disparo de evento sintético na SPA do *virtual-ms-java*, usado para reproduzir cenários de ativação *Constructive*.

4.4.4 Alagamento e Auditoria de Alertas

A fonte CGESP, ingerida por *web scraping*, está habilitada a produzir eventos de alagamento que alimentariam o Gêmeo Digital e poderiam acionar missões de verificação pelo drone *Constructive*. Como nenhum alagamento foi reportado pelo CGESP durante o período de validação da plataforma, o cenário permanece arquiteturalmente suportado e reproduzível via o disparo sintético da SPA (Subseção 4.4.3). Paralelamente, eventos de colisão (*batida*) durante o voo simulado são capturados pelo *Event Hub* e entram no pipeline de auditoria, compondo o quadro completo de observabilidade descrito na Subseção 4.5.

4.5 Validação e Observabilidade

A consolidação dos componentes desenvolvidos foi exercitada operacionalmente em três frentes:

- Barramento de Eventos (*Event Hub*): a aplicação *Event Hub* (desenvolvida em Rust), crucial para o padrão *Publish/Subscribe* da arquitetura, foi modificada e containerizada em Docker, atendendo aos requisitos de Reprodutibilidade e facilidade de *deployment*. A persistência de eventos foi implementada sobre quatro bases segregadas: MariaDB *eventhub* (CQRS-Write) e *eventhub_read* (CQRS-Read), além do InfluxDB com *buckets events* (escrita) e *projections* (leitura para Grafana), eliminando a dependência de volumes Docker para durabilidade.

- Módulo **proto-shared**: para garantir a Interoperabilidade entre os diversos componentes LVC, foi desenvolvido e validado o módulo **proto-shared**. Este artefato, estruturado em Apache Maven e sem código-fonte próprio (código-fonte zero), automatiza a compilação do IDL (*Protocol Buffers*), gerando *stubs* de comunicação em Java, Python e Rust a partir de um único contrato canônico **.proto**. A geração para Java e Python é realizada pelo plugin **ascope protobuf-maven-plugin** (SCOPES, 2026) integrado ao Apache Maven, enquanto os *stubs* Rust são produzidos por uma imagem Docker dedicada com **protoc-gen-prost** e **protoc-gen-tonic**, garantindo a padronização e a consistência das interfaces de integração entre os componentes LVC.
- Adaptadores de Ingestão do *Live*: os três adaptadores do **live-ms-java** (*AisAdapter*, *OlhoVivoAdapter* e *CgespAdapter*) foram implementados e verificados operacionalmente, demonstrando a capacidade de ingerir dados geoespaciais de fontes reais heterogêneas (AIS, SPTrans, CGESP) e publicá-los como eventos padronizados no *Event Hub*, fechando o ciclo *Live-para-Plataforma*.

A observabilidade do comportamento da plataforma é fornecida por dois *dashboards* Grafana, consumidos a partir do caminho de leitura CQRS-Read (*bucket projections* do InfluxDB e base **eventhub_read** da MariaDB), que consolidam a operação em painéis complementares.

Dois painéis Grafana (Figura 5) consolidam a observabilidade da plataforma a partir do caminho de leitura CQRS-Read. O painel superior oferece a visão operacional — entidades monitoradas por área, contagens de *check-in* e *check-out*, mapa interativo com histórico de entidades e *log* de auditoria — evidenciando o fluxo *Live* para *Virtual* e a segregação de leitura viabilizada pelo CQRS. O painel inferior concentra a auditoria de alertas (alagamentos reportados pelo CGESP e colisões detectadas na simulação do drone) e a observabilidade das fotos capturadas pelo *Constructive* a partir das coordenadas dos POIs (exemplificado pelo ponto de ônibus *Caio Prado C/B*), fechando o ciclo *Live-Virtual-Constructive*-Observabilidade sobre um único plano de controle.



Figura 5 – Painéis Grafana de observabilidade da plataforma LVC. **Painel superior:** visão operacional com entidades por área, *check-in/check-out*, mapa histórico e *log* de auditoria. **Painel inferior:** auditoria de alertas (alagamentos do CGESP e colisões do drone simulado) e observabilidade das fotos capturadas pelo *Constructive*.

4.6 Discussão

Os resultados obtidos validam a viabilidade técnica da arquitetura em quatro dimensões fundamentais. Primeiro, a rastreabilidade: o *dashboard* de auditoria (Figura 5, painel inferior) reconstrói a sequência completa de eventos que dispararam cada missão *Constructive* — do *check-in* da entidade *Live* no POI à publicação das fotos capturadas pelo drone —, com cada evento persistido de forma imutável no InfluxDB e na Mari-aDB (via *Transactional Outbox*) e referenciável por *timestamp* e identificador, permitindo *replay* integral do cenário conforme prevê o padrão *Event Sourcing*. Segundo, a reprodutibilidade: a containerização completa do *Event Hub* e dos microsserviços garante que qualquer pesquisador possa reproduzir o ambiente de execução com um único comando. Terceiro, a interoperabilidade multi-linguagem: o módulo **proto-shared** comprovou a geração bem-sucedida de *stubs* em Java, Python e Rust a partir de um único contrato **.proto**, confirmando que componentes implementados em linguagens heterogêneas podem integrar-se de forma transparente e fortemente tipada. Quarto, a integração com o mundo real: os três adaptadores do **live-ms-java** demonstraram a capacidade de ingerir dados geoespaciais de fontes reais (AIS, SPTrans, CGESP) e publicá-los como eventos padronizados no barramento, fechando o ciclo *Live-para-Plataforma*; a diversidade de naturezas de integração — REST autenticada, *web scraping* e API com cadastro — sobre um único contrato canônico constitui evidência empírica da facilidade de integração arquitetural prevista no Objetivo 4.

Os padrões adotados também se distinguem em pontos específicos das alternativas tradicionais. O *Event Sourcing*, ao contrário do modelo de federação HLA (em que o estado da simulação é volátil e distribuído entre federados), provê um *log* imutável e centralizado que simplifica depuração e auditoria. A reatividade da plataforma, por sua vez, opera no nível arquitetural (barramento assíncrono e *streaming* gRPC) e não no nível de *framework*: optou-se por não utilizar Spring Boot WebFlux ou Project Reactor, pois o ecossistema gRPC já provê I/O não-bloqueante nativo sobre HTTP/2 e as *Virtual Threads* (Java 25) oferecem concorrência escalável com modelo imperativo simples, evitando a complexidade de *streams* reativos (Mono/Flux) e facilitando a depuração. Complementarmente, o uso de *Scoped Values* (JEP 506) para propagar contexto entre *Virtual Threads* substitui **ThreadLocal**, problemático em ambientes de alta concorrência.

Esses achados dialogam diretamente com a literatura. A arquitetura proposta endereça as limitações de HLA e DIS identificadas por Gao et al. (2021) e Dani et al. (2023) (custo de licenciamento, curva de aprendizado e baixa integração com o ecossistema moderno), ao substituí-las por um conjunto de tecnologias *Open Source* maduras e amplamente adotadas. O *Event Sourcing*, conforme argumentado por Debski et al. (2018) e Overeem, Spoor e Jansen (2017), provê rastreabilidade e reprodutibilidade que não existem no modelo de federação HLA, onde o estado é volátil e distribuído. O CQRS, por sua vez, permite a segregação de preocupações entre ingestão de alta frequência (*Live/Constructive*) e consultas de baixa latência (*dashboards/Virtual*), alinhando-se aos princípios do *Reactive Manifesto* (DEBSKI et al., 2018).

O caráter *Open Source* da plataforma é uma contribuição em si: ao disponibilizar integralmente o código-fonte e a documentação no Deepwiki, este trabalho oferece uma alternativa acessível e de custo zero às arquiteturas proprietárias tradicionais, eliminando a barreira financeira que historicamente limitou a pesquisa em simulação LVC a instituições com orçamento para licenciamento de RTI.

Reconhecem-se, contudo, limitações inerentes ao escopo deste estudo de viabi-

lidade técnica. A validação estatística comparativa entre dados *Live* e modelos *Virtual/Constructive*, essencial para comprovar a fidelidade dos Gêmeos Digitais, permanece como trabalho futuro, dependente da integração completa do ciclo de simulação. Similarmente, os testes foram conduzidos em máquina única, e a escalabilidade horizontal do barramento sob carga massiva de eventos não foi avaliada. Estas limitações não invalidam os achados, mas delimitam seu alcance: o que se demonstrou foi a viabilidade arquitetural e de integração, não a acurácia preditiva dos modelos ou o desempenho sob condições extremas de carga.

5 Conclusão

Este trabalho apresentou o projeto, a implementação e a validação de uma arquitetura orientada a eventos para uma plataforma de simulação LVC aplicada ao contexto de *Smart Cities*, utilizando exclusivamente tecnologias *Open Source*. Os quatro objetivos estabelecidos na Subseção 3.1 foram atendidos: a revisão da literatura fundamentou a escolha dos padrões *Event Sourcing*, CQRS e dos princípios reativos; a arquitetura foi definida com um contrato canônico via *Protocol Buffers* e gRPC; a implementação foi materializada em seis módulos independentes (*proto-shared*, *event-hub-ms-rust*, *live-ms-java*, *constructive-ms-python*, *virtual-ms-java* e *infra-local-shared*); e a validação exercitou o contrato sobre múltiplas fontes reais e um simulador.

A plataforma não foi concebida como instância de um cenário urbano específico, mas como base extensível para integração de fontes diversas (APIs públicas, *web scraping*, dados geoespaciais abertos e simuladores), todas de acesso público e gratuito. Os cenários de interação LVC demonstrados (Subseção 4.4) fecham esse ciclo: o cadastro de áreas e POIs no Gêmeo Digital *Virtual*, a ingestão contínua de entidades *Live* com emissão de *check-in* e *check-out* por área, o disparo de missões do drone *Constructive* a partir de eventos reais (como o ônibus no ponto *Caio Prado C/B*) com captura e auditoria de fotos, e a propagação de alertas de colisão do drone e de alagamento (quando reportado pelo CGESP) para a camada de observabilidade no Grafana.

As contribuições deste trabalho são quatro: (i) uma arquitetura *event-driven Open Source* para simulação LVC que substitui HLA/DIS sem sacrificar interoperabilidade, atacando diretamente as barreiras de custo e complexidade identificadas por Gao et al. (2021) e Dani et al. (2023); (ii) o módulo *proto-shared* como ponto único de verdade contratual, validando a geração automatizada de *stubs* multi-linguagem (Java, Python, Rust) a partir de um único arquivo *.proto*; (iii) a integração de três fontes *Live* reais e públicas (AIS, SPTrans e CGESP) como evidência empírica da capacidade de ingestão heterogênea sob um contrato canônico; e (iv) cenários LVC concretos que exercitam o ciclo *Live–Virtual–Constructive–Observabilidade* ponta-a-ponta sobre o campus Higienópolis da UPM.

O estudo de viabilidade técnica, objetivo central deste trabalho, foi cumprido: demonstrou-se que é possível alcançar interoperabilidade LVC com ferramentas acessíveis, sem sacrificar desempenho, rastreabilidade ou reprodutibilidade. A plataforma estabelece fundações para um ecossistema de simulação urbana aberto e extensível, representando um passo relevante para democratizar a pesquisa em Gêmeos Digitais e simulação no contexto de *Smart Cities*. Como trabalhos futuros, destacam-se a validação estatística comparativa entre dados *Live* e modelos *Virtual/Constructive* e a avaliação da escalabilidade horizontal do barramento sob carga massiva de eventos.

6 Referências bibliográficas

- AIS Friends. *AIS Friends API Documentation*. 2025. Disponível em: <<https://www.aisfriends.com/docs/api/v1>>. Acesso em: 2026-05-05.
- BANKS, J. (Ed.). *Handbook of Simulation: Principles, Methodology, Advances, Applications, and Practice*. 1. ed. Wiley, 1998. ISBN 9780471134039 9780470172445. Disponível em: <<https://onlinelibrary.wiley.com/doi/book/10.1002/9780470172445>>. Acesso em: 2025-10-23.
- Centro de Gerenciamento de Emergências (CGE). *Dados de Alagamentos e Monitoramento Meteorológico*. Disponível em: <<https://www.cgesp.org/v3/index.jsp>>. Acesso em: 2026-05-03.
- Cognition Labs. *DeepWiki*. 2025. Disponível em: <<https://deepwiki.com>>. Acesso em: 2026-05-04.
- DAHMAN, J. High Level Architecture for simulation. In: *Proceedings First International Workshop on Distributed Interactive Simulation and Real Time Applications*. Eilat, Israel: IEEE Comput. Soc. Press, 1997. p. 9–14. ISBN 9780818677731. Disponível em: <<http://ieeexplore.ieee.org/document/568652/>>. Acesso em: 2025-08-20.
- DANI, A. A. H.; SUPANGKAT, S. H.; LUBIS, F. F.; NUGRAHA, I. G. B. B.; KINANDA, R.; RIZKIA, I. Development of a Smart City Platform Based on Digital Twin Technology for Monitoring and Supporting Decision-Making. *Sustainability*, v. 15, n. 18, p. 14002, set. 2023. ISSN 2071-1050. Disponível em: <<https://www.mdpi.com/2071-1050/15/18/14002>>. Acesso em: 2025-11-05.
- DEBSKI, A.; SZCZEPANIK, B.; MALAWSKI, M.; SPAHR, S.; MUTHIG, D. A Scalable, Reactive Architecture for Cloud Applications. *IEEE Software*, v. 35, n. 2, p. 62–71, mar. 2018. ISSN 0740-7459, 1937-4194. Disponível em: <<https://ieeexplore.ieee.org/document/7950867/>>. Acesso em: 2025-08-20.
- GAO, Y.; ZHANG, Y.; ZHOU, X.; LU, H. Overview of Simulation Architectures Supporting Live Virtual Constructive (LVC) Integrated Training. In: *2021 6th International Conference on Control, Robotics and Cybernetics (CRC)*. Shanghai, China: IEEE, 2021. p. 333–338. ISBN 9781665424370. Disponível em: <<https://ieeexplore.ieee.org/document/9620135/>>. Acesso em: 2025-08-20.
- GRIEVES, M. *Digital Twin: Manufacturing Excellence through Virtual Factory Replication*. [S.l.], 2014. 8 p. Disponível em: <<https://www.3ds.com/fileadmin/PRODUCTS-SERVICES/DELMIA/PDF/Whitepaper/DELMIA-APRISO-Digital-Twin-Whitepaper.pdf>>. Acesso em: 2025-10-30.
- KIM, K.; PARK, T.; PASTRANA, J.; MARIN, M.; CORTES, E. A.; RABELO, L. C.; LEE, G. Modeling of complex scenarios using LVC simulation. In: *Proceedings of the Winter Simulation Conference 2014*. Savannah, GA, USA: IEEE, 2014. p. 2931–2941. ISBN 9781479974863 9781479974849. Disponível em: <<http://ieeexplore.ieee.org/document/7020133/>>. Acesso em: 2025-08-20.
- LJUBISAVLJEVIĆ, M.; TURAJLIĆ, N.; PETROVIĆ, M.; FURTULA, F.; BELČA, V. A Comparison of Event Sourcing Implementations: Using Apache Kafka Streams APIs and
- Campus Higienópolis: Rua da Consolação, 930 - Consolação - São Paulo - SP - CEP 01302-907
Tel. (11) 2114-8301 - www.mackenzie.br - e-mail: fci@mackenzie.br

Microsoft Service Fabric Actor Services. In: *2024 23rd International Symposium INFOTEH-JAHORINA (INFOTEH)*. East Sarajevo, Bosnia and Herzegovina: IEEE, 2024. p. 1–6. ISBN 9798350329940. Disponível em: <<https://ieeexplore.ieee.org/document/10495980/>>. Acesso em: 2025-08-20.

LONG, Z. Improvement and Implementation of a High Performance CQRS Architecture. In: *2017 International Conference on Robots & Intelligent System (ICRIS)*. Huai An City, China: IEEE, 2017. p. 170–173. ISBN 9781538612279. Disponível em: <<http://ieeexplore.ieee.org/document/8101372/>>. Acesso em: 2025-08-20.

LOPER, M.; SITTERLE, V. Evolving Lvc To Include Evaluation Of Human-Ai Teaming Dynamics. In: *2023 Winter Simulation Conference (WSC)*. San Antonio, TX, USA: IEEE, 2023. p. 2506–2517. ISBN 9798350369663. Disponível em: <<https://ieeexplore.ieee.org/document/10408209/>>. Acesso em: 2025-08-20.

LV, Z.; SHANG, W.-L.; GUIZANI, M. Impact of Digital Twins and Metaverse on Cities: History, Current Situation, and Application Perspectives. *Applied Sciences*, v. 12, n. 24, p. 12820, dez. 2022. ISSN 2076-3417. Disponível em: <<https://www.mdpi.com/2076-3417/12/24/12820>>. Acesso em: 2025-11-05.

MAZZETTO, S. A Review of Urban Digital Twins Integration, Challenges, and Future Directions in Smart City Development. *Sustainability*, v. 16, n. 19, p. 8337, set. 2024. ISSN 2071-1050. Disponível em: <<https://www.mdpi.com/2071-1050/16/19/8337>>. Acesso em: 2025-11-05.

Microsoft Corporation. *AirSim*. Microsoft Corporation, 2022. Disponível em: <<https://github.com/microsoft/airsim>>. Acesso em: 2026-05-05.

NOSEWORTHY, J. R. The Test and Training Enabling Architecture (TENA) Supporting the Decentralized Development of Distributed Applications and LVC Simulations. In: *2008 12th IEEE/ACM International Symposium on Distributed Simulation and Real-Time Applications*. [S.l.: s.n.], 2008. p. 259–268.

OpenStreetMap contributors. *OpenStreetMap*. 2026. Disponível em: <<https://www.openstreetmap.org>>. Acesso em: 2026-05-03.

OVEREEM, M.; SPOOR, M.; JANSEN, S. The dark side of event sourcing: Managing data conversion. In: *2017 IEEE 24th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. Klagenfurt, Austria: IEEE, 2017. p. 193–204. ISBN 9781509055012. Disponível em: <<http://ieeexplore.ieee.org/document/7884621/>>. Acesso em: 2025-08-20.

POTTEIGER, B.; DIGNAN, T.; MILLS, A.; PAVELKA, E.; FREY, C.; NATHAN, B.; DAGNE, M.; GARIBALDI, V.; OTTER, B. Live Virtual Constructive Environment for Assuring the Safety and Security of Complex Autonomous Vehicles. In: *2023 IEEE International Conference on Assured Autonomy (ICAA)*. Laurel, MD, USA: IEEE, 2023. p. 53–56. ISBN 9798350326017. Disponível em: <<https://ieeexplore.ieee.org/document/10207574/>>. Acesso em: 2025-08-20.

SCOPES, A. *Protobuf Maven Plugin*. 2026. Disponível em: <<https://ascopes.github.io/protobuf-maven-plugin/index.html>>. Acesso em: 2025-09-12.

SILVA, F. G. B. O. D.; OLIVEIRA, M. G. D.; VOGEL, M. H. M.; TAKASE, F. K.; LIMA, B. L. S.; SAITO, L. A. M. Comparing MQTT and gRPC Protocols Performance in a IoT Smart Campus Cybersecurity Scenario. In: *2026 IEEE 17th Latin America Symposium on Circuits and System (LASCAS)*. Arequipa, Peru: IEEE, 2026. p. 1–5. ISBN 9798331570972. Disponível em: <<https://ieeexplore.ieee.org/document/11457148/>>. Acesso em: 2026-05-05.

São Paulo Transporte S/A. *API Olho Vivo: Guia de Referência*. Disponível em: <<https://www.sptrans.com.br/desenvolvedores/api-do-olho-vivo-guia-de-referencia>>. Acesso em: 2026-05-03.

THEUNISSEN, E.; KOTEGAWA, T. Applying LVC to testing and evaluation of DAA systems. In: *2017 IEEE/AIAA 36th Digital Avionics Systems Conference (DASC)*. St. Petersburg, FL: IEEE, 2017. p. 1–7. ISBN 9781538603659. Disponível em: <<http://ieeexplore.ieee.org/document/8102089/>>. Acesso em: 2025-08-20.

VARSHNEY, M.; PICKETT, K.; BAGRODIA, R. A Live-Virtual-Constructive (LVC) framework for cyber operations test, evaluation and training. In: *2011 - MILCOM 2011 Military Communications Conference*. [S.l.: s.n.], 2011. p. 1387–1392.